

3.1.7 Activity: Word Frequency Counter

جملے کو .split() سے الفاظ کی فہرست بناتے ہیں، پھر ہر لفظ ڈکشنری میں کلید ہوتا ہے اور اس کی گنتی قدر۔

CS XII

Chapter 3: Programming Fundamentals

Lecture 08 of 17

3.1.7 Activity: Word Frequency Counter

LEARN IT

Count how many times each word appears in a sentence. This is a classic use of a **dictionary** plus `.get()`: the word is the key, the count is the value.

```
sentence = "python is easy python is fun python is powerful"
words = sentence.split()
word_freq = {}
for word in words:
    word_freq[word] = word_freq.get(word, 0) + 1
print(f"Sentence: '{sentence}'\n")
print("Word Frequency Table:")
for word, count in word_freq.items():
    print(f"{word:<10} : {count}")
top_word = max(word_freq, key=word_freq.get)
print(f"\nMost Frequent: '{top_word}' ({word_freq[top_word]} times)")
```

WORKED EXAMPLE

Logic, step by step.

1. `sentence.split()` cuts on spaces and gives
`['python', 'is', 'easy', 'python', 'is', 'fun', 'python', 'is', 'powerful']`.
2. For a new word, `word_freq.get(word, 0)` is 0, then you store $0 + 1 = 1$.
3. For a word already in the dictionary, `.get(word, 0)` returns the current count, then you add 1.
4. `{word:<10}` pads the word to 10 characters so the colons line up.
5. `max(word_freq, key=word_freq.get)` picks the key whose value is largest. If two words share the top count, `max` returns the first of those keys in insertion order.

OUTPUT

Sentence: 'python is easy python is fun python is powerful'

Word Frequency Table:

```
python      : 3
is          : 3
easy        : 1
fun         : 1
powerful    : 1
```

Most Frequent: 'python' (3 times)

After seeing	<code>word_freq</code>
python	<code>{'python': 1}</code>
is	<code>{'python': 1, 'is': 1}</code>
easy	<code>{'python': 1, 'is': 1, 'easy': 1}</code>
python (again)	<code>{'python': 2, 'is': 1, 'easy': 1}</code>
... end of sentence	<code>{'python': 3, 'is': 3, 'easy': 1, 'fun': 1, 'powerful': 1}</code>

Here `python` and `is` both occur 3 times. `max` returns `'python'` because it appears first in the dictionary.

REMEMBER IT

The heart of every counter is one line: `d[item] = d.get(item, 0) + 1`. You can count letters, roll numbers or votes with the same pattern.

COMMON MISTAKE

This program is case-sensitive and space-based. "Python" and "python" would be two keys. Punctuation such as `easy.` would not match `easy`. For a class exercise, keep the sentence lowercase with spaces only, as here. If you wrote `word_freq[word] += 1` without `.get` on a new word, you would get `KeyError`.

اردو میں سمجھیں

جملے کو `.split()` سے الفاظ کی فہرست بناتے ہیں، پھر ہر لفظ ڈکشنری میں کلید ہوتا ہے اور اس کی گنتی قدر۔ نئے لفظ کے لیے `get(word, 0) + 1` سے گنتی 1 سے شروع ہوتی ہے۔ `max(... , key=word_freq.get)` سب سے زیادہ بار آنے والا لفظ دیتا ہے۔

CHECK YOURSELF

1. How many times does "is" occur in the sample sentence?
2. Why use `.get(word, 0)` instead of `word_freq[word]` on the right-hand side?

Answers: 1. Three times. 2. Because a brand-new word is not yet a key; `.get` returns 0 instead of raising `KeyError`.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Word frequency uses a dict as:

- a. word → count
- b. count → word only
- c. A stack of letters
- d. A queue of files

MCQ 2. A standard pattern is:

- a. $d[w] = d.get(w,0)+1$
- b. sort first always $O(n^2)$
- c. binary search the sentence
- d. draw OSI

MCQ 3. You usually:

- a. `split()` and maybe `lower()`
- b. Keep punctuation attached always
- c. Use a set so counts die
- d. Use a tuple of counts you cannot update

MCQ 4. `split("a a b")` has length:

- a. 3
- b. 2
- c. 1
- d. 0

MCQ 5. This activity shows:

- a. Dicts + loops + strings
- b. Only Access reports
- c. Only NAND
- d. Only HCI

MCQ 6. Most frequent word is:

- a. The key with max count
- b. Always the first word
- c. Always the last
- d. A foreign key

Extra short questions

1. Write the counting pattern.
2. How to print nicely.
3. A limitation of naive split.
4. Why a dict not a list of pairs?

Extra long question

1. Count words in "to be or not to be". Show the dict, then the mode word(s). Full code.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. `for w in text.lower().split(): d[w]=d.get(w,0)+1`

2. `for k,v in d.items(): print(k,v)` or sorted by count.
3. Punctuation ("hello,") is a different word.
4. $O(1)$ average update by word.

Long

1. to:2 be:2 or:1 not:1. Bimodal to/be. Code with `get`. Mention `lower()`.

Next lecture: 3.2 Functions in Python